AMD Radeon GPU / ROCm Application Development

ArtisanAI

Real-time Stylized Image Generation System Based on SDXL + ControlNet

Team: **Rui Yi Jin Qu** · Participants: Ruiqing He, Yi Gao

Project Overview Document | Version v1.0 | August 6, 2026

1. Project Background

1.1 Industry Background

Diffusion models, represented by Stable Diffusion, have become the mainstream technical approach for AIGC image generation. Among them, Stable Diffusion XL (SDXL), with its native 1024×1024 resolution and significantly improved image quality, has become the widely adopted base model in the open-source community. However, the iterative sampling mechanism of diffusion models inherently makes them "slow". Standard SDXL requires 30 to 50 denoising steps, and generating a single image often takes tens of seconds, which makes interactive use impractical. Meanwhile, the demand for image generation is increasingly shifting from text-to-image toward image-to-image. Users often have existing photos or sketches and seek to modify the artistic style while preserving the original composition and structure, rather than generating an entirely unrelated image from noise. This trend places greater emphasis on the structural controllability of the model.

1.2 Problems to Solve

Problem	Specific Manifestation
slow inference speed	Standard SDXL requires 30 to 50 sampling steps, and the integration of ControlNet can further double the cost of each step, which ultimately undermines the interactive experience.
uncontrollable structure	Pure image-to-image relies solely on `strength` for control. If strength is too high, the original contours are lost; conversely, the style of the original image barely changes.
high style-switching cost	Switching between style LoRAs requires loading weights from disk each time, which takes several seconds and precludes rapid comparison.
large VRAM usage	A naive implementation combining SDXL with full-size ControlNet and multiple LoRAs can easily exceed the VRAM capacity of a single GPU.

1.3 Project Positioning

ArtisanAI targets the AMD Radeon GPU and ROCm platform, aiming to build an image stylization system that features **structural controllability, flexible style switching, and interactive responsiveness**. The system is built upon three core technical components:

- **SDXL-Lightning adversarial distillation LoRA** compresses sampling steps from 50 to 8, effectively alleviating the inference speed issue.
- **Distilled Canny ControlNet** employs edge maps to guide the generation process, ensuring structural consistency with the input. The distilled 160M-parameter variant is adopted in place of the 1.25B full-size version, significantly reducing computational overhead.
- **Multi-LoRA VRAM-resident preloading with dynamic weight blending** loads all eight style LoRAs into GPU memory at startup; switching between styles only updates the active adapter weights without triggering disk I/O, thereby eliminating per-switch latency.

2. Target Users and Application Scenarios

2.1 Target Users

User Group	Core Needs	System Capabilities
Self-media / Content Creators	Quickly convert source images into multiple styled illustrations for covers, social media posts	One input image, 8 styles switchable in sub-second for comparison, ready to use
Visual Designers	Rapidly explore style directions during the concept phase, avoiding premature investment in a single approach	Switch styles under a fixed seed: composition remains unchanged, only the style varies, enabling controlled comparison
E-commerce / Marketing Professionals	Stylize product images and promotional materials, reducing outsourcing costs for illustrations	Edge constraints ensure product contours remain intact; style overlay does not alter subject structure
Education / Demonstration	Intuitively demonstrate diffusion models, ControlNet, and LoRA to non-technical audiences	Web UI exposes all parameters and displays real-time per-stage timing breakdown
General Users	Photo artification and avatar stylization, with zero technical barrier	Open in browser and use immediately; style switching auto-fills recommended prompts

2.2 Typical Application Scenarios

Single-image Multistyle Rapid Comparison

The user uploads a photo with a fixed random seed `Seed = 42` and a fixed redraw intensity `Strength = 0.7`, then sequentially switches through four styles: Anime, Studio Ghibli, Oil Painting, and Pixar. Because the random seed and structural constraints remain unchanged, all four outputs retain the exact same composition, with only the artistic style varying across results. Since style switching does not involve reloading weights, each switch only costs a single inference pass.

Structure Preservation Strength Tuning

The user adjusts two independent parameters, ControlNet Scale (controlling edge constraint strength) and Strength (controlling redraw intensity), to continuously navigate between "faithful to the original image" and "strong stylization", thereby finding the optimal balance for the given input.

Performance Demonstration and Tuning

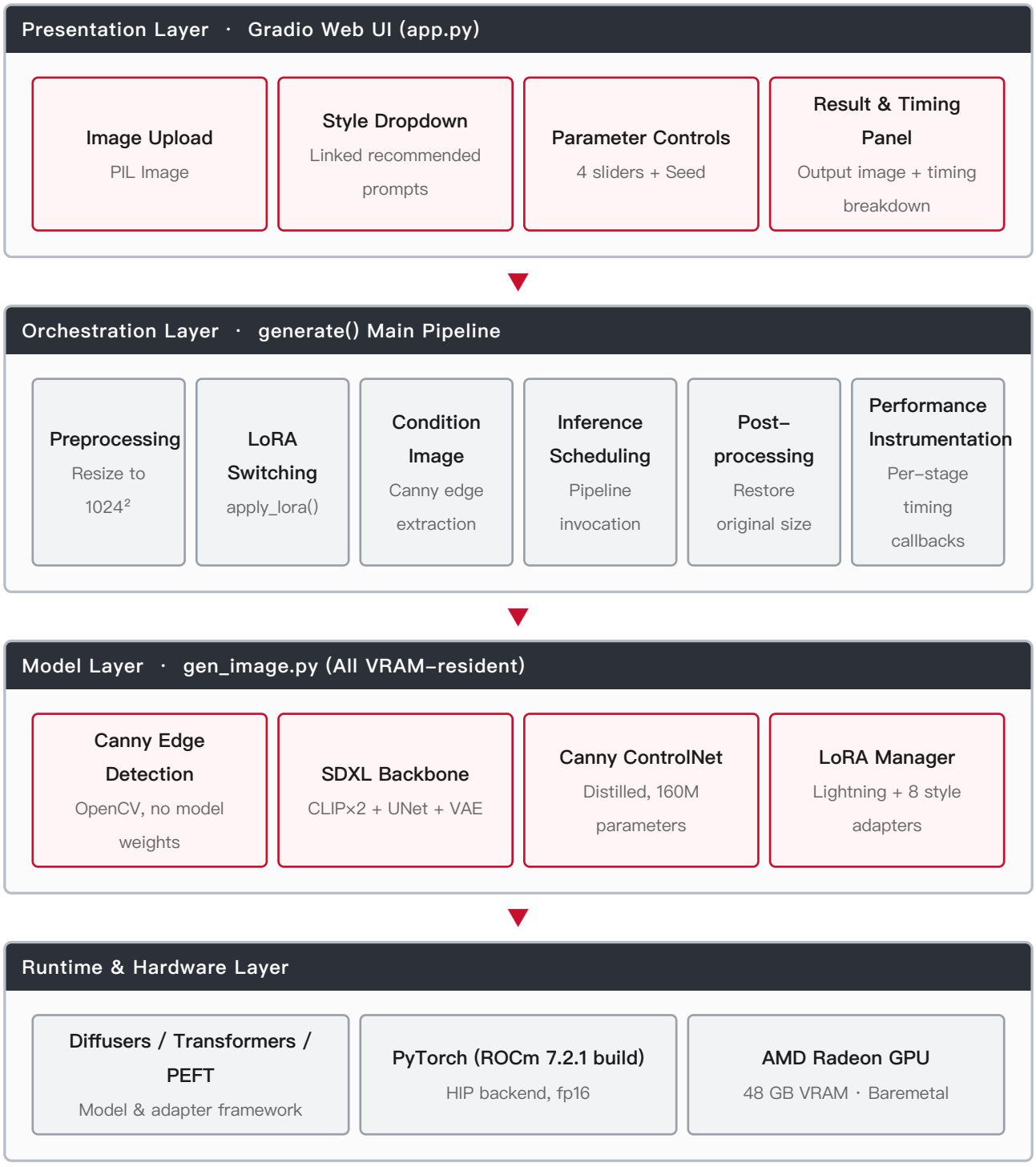
The interface displays real-time timing breakdowns for each stage, including edge extraction, text encoding, step-by-step denoising, and VAE decoding. This facilitates bottleneck identification and optimization validation on the AMD platform.

2.3 Scenario Constraints

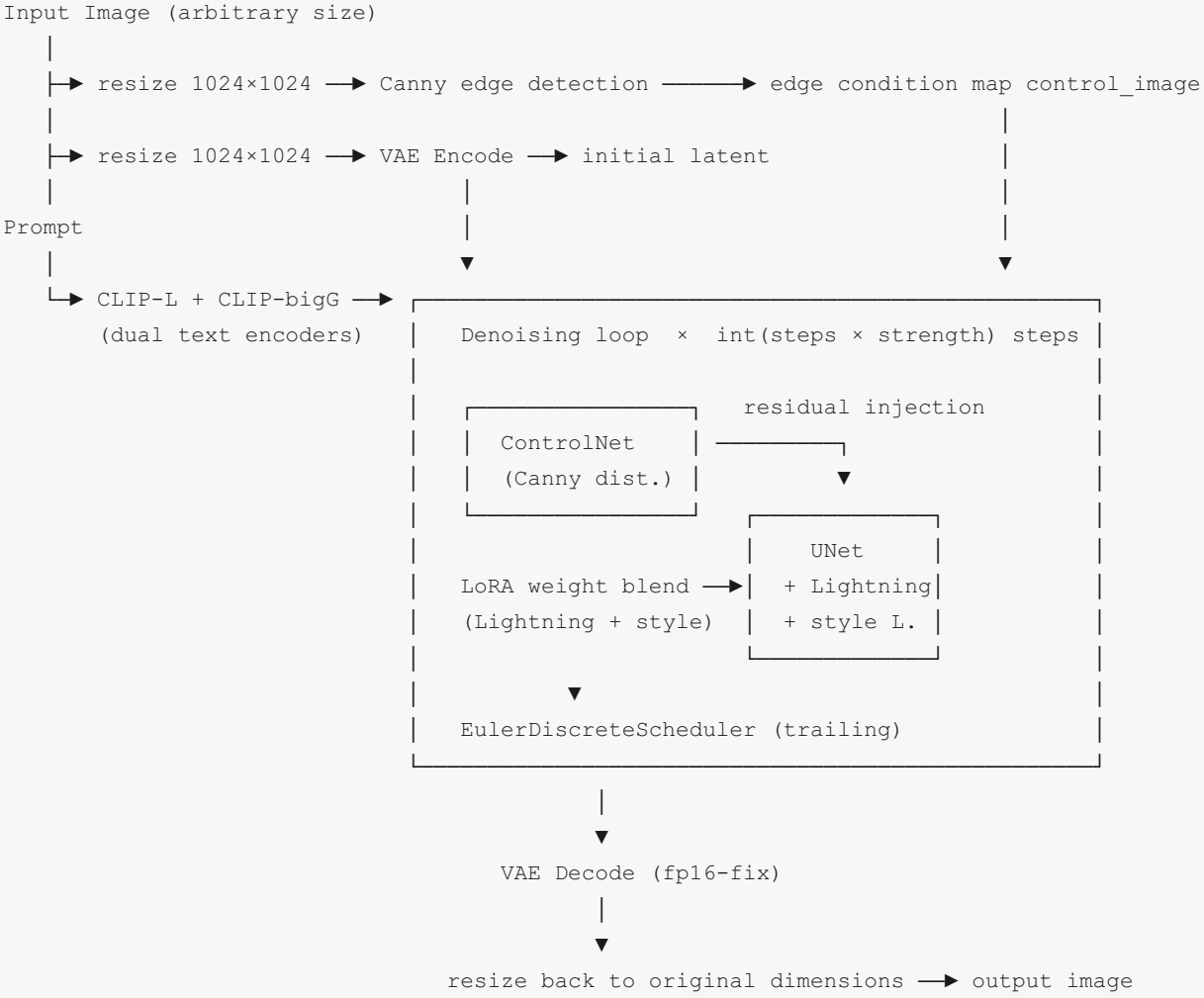
> This system is designed specifically for **image–to–image stylization**, requiring an input image and does not support text–to–image generation. It produces outputs at 1024×1024 resolution, which are then resized to match the original input dimensions. Currently, the system processes single images on demand and does not support batch queuing.

3. System Architecture

3.1 Layered Architecture



3.2 Inference Data Flow



About actual step count: In image-to-image mode, the actual number of denoising steps executed is `int(num_inference_steps × strength)`. With the interface defaults `steps=8` and `strength=0.99`, **7 steps** of denoising are actually performed. This is the combined effect of the SDXL-Lightning distilled LoRA and the image-to-image mechanism, and is the direct reason this system achieves interactive-level responsiveness.

3.3 Key Design Decisions

Decision	Approach	Benefit
Full LoRA preloading	At startup, `preload_style_loras()` loads all 8 style adapters into VRAM simultaneously. Switching styles simply calls `set_adapters()` to update the weights.	Style switching overhead drops from seconds (due to disk I/O and loading) to near–zero latency. The additional VRAM cost is minimal, totaling only ~72 MB (8 × 9 MB).
Dual LoRA weight blending	The acceleration Lightning LoRA remains active at a fixed weight of 1.0, while the style LoRA is stacked as a secondary adapter.	This decouples acceleration from style, ensuring that every style benefits from the 8–step acceleration without conflict.
Startup warmup	The system executes one full inference pass with a built–in sample image upon service startup.	This shifts the overhead of first–time GPU kernel compilation from the user's first request to the startup phase, ensuring a faster initial response for users.
Full fp16, no offload	All components reside permanently in VRAM, and `enable_model_cpu_offload()` is intentionally disabled.	Avoiding Host↔Device weight transfer at each inference step comes at the cost of higher VRAM usage.
Edge map caching	The system caches the most recent Canny edge map using the input image's content hash as the key.	This skips the edge detection step entirely when tuning parameters or switching styles on the same image, which significantly speeds up the primary interactive workflow.
Per–stage performance instrumentation	We record step–by–step timing via `callback_on_step_end` with GPU synchronization and collect PyTorch VRAM peak metrics.	This enables precise bottleneck identification. Note that this can be disabled in production to eliminate synchronization overhead.

4. Models and Algorithms

4.1 Model Inventory

Role	Model	Description
Base Model	`stabilityai/stable-diffusion-xl-base-1.0` (fp16 variant)	SDXL 1.0 featuring dual text encoders and a 2.6B-parameter UNet, optimized for native 1024×1024 resolution.
Structure Control	`diffusers/controlnet-canny-sdxl-1.0-small`	A distilled SDXL ControlNet with 160M parameters (320 MB in fp16), approximately 1/7.8 the size of the full version.
Edge Detection	OpenCV Canny (wrapped by `controlnet-aux`)	A classic algorithm that requires no model weights, utilizing CPU-only inference with zero VRAM overhead.
Image Decoder	`madebyollin/sdxl-vae-fp16-fix`	A patched VAE designed to prevent numerical overflow issues often found in the official SDXL VAE when running in fp16.
Acceleration Module	`ByteDance/SDXL-Lightning` `sdxl_lightning_8step_lora.safetensors`	An 8-step sampling LoRA achieved via progressive adversarial distillation.
Style Modules	`ntc-ai/SDXL-LoRA-slider.*` × 8	See Section 4.4. Each weight file is only 9 MB.

4.2 Canny ControlNet Structural Constraint

ControlNet duplicates the UNet encoder branch and accepts an additional spatial condition map. It then injects condition features as residuals into the corresponding layers of the backbone UNet, imposing spatial constraints without damaging the base model's capabilities.

Choice of Condition Type

Structural condition types for image generation can be arranged along a spectrum of locking strength, where Canny's binary hard edges represent the tightest constraint, Depth's 3D hierarchy offers the most in-plane freedom, and options like Lineart, SoftEdge, and Scribble occupy the intermediate positions. This spectrum reflects a fundamental trade-off: composition fidelity improves with tighter locking, but stylization freedom diminishes accordingly.

The selection for this project underwent a measurement-driven adjustment. The initial version adopted SoftEdge, which combines the HED detector with the full-size SoftEdge ControlNet. This choice theoretically offers the best balance by sitting in the middle of the locking spectrum. However, this path encountered a hard engineering constraint. The open-source community provides no distilled small models for the SoftEdge or Scribble types. The only available weights are the 1.25B-parameter full-size ControlNets, and the repository used only supplies the fp32 format, which occupies 5 GB and requires on-the-fly precision conversion at load time. The final switch to Canny was driven by a decisive factor. Canny and Depth are the only two condition types with official distilled versions. Among these, Canny shares the closest semantics with SoftEdge, as both belong to the same edge family, making it the only viable downgrade replacement. The benefits of the switch are summarized below:

Dimension	SoftEdge (original)	Canny (current)	Change
ControlNet parameters	1.25 B	**160 M**	↓ **7.8×**
Weight file size	5004 MB (fp32)	**320 MB** (fp16)	↓ **15.6×**
Edge detector	HED, 29.4 MB weights	**OpenCV, no weights**	↓ 100%

The cost is that Canny outputs binary hard edges, preserving image noise as well, which can produce a somewhat stiff outline effect under heavy stylization. In practice, this is mitigated by appropriately lowering `controlnet_conditioning_scale` (0.3—0.4), which is precisely why this parameter is exposed in the interface.

Constraint strength is controlled by `controlnet_conditioning_scale` (UI default 0.4): higher values more closely adhere to the original edges, lower values allow freer stylization.

> **On the practical benefit of "7.8× parameter reduction":** ControlNet is a bypass branch attached to the UNet, not a replacement. Relative to SDXL's 2.6B UNet, the full-size ControlNet increases per-step computation to about 1.48×; switching to the 160M distilled version reduces it to about 1.06×, so **the actual speedup in the denoising stage is approximately 28%, not 7.8×**. The true computational bulk is always the UNet itself. This project maintains a clear understanding of this: ControlNet lightweighting is a valuable piece, but the compression of sampling steps (see Section 4.3) is the order-of-magnitude optimization.

4.3 SDXL-Lightning Few-step Sampling

SDXL-Lightning employs **progressive adversarial distillation**: using the original SDXL as the teacher model, a student model is trained to approximate the teacher's multi-step sampling result in just a few steps, with an adversarial loss introduced to avoid the blurring common in few-step sampling. The output is released as a LoRA that can be directly applied on top of the original SDXL weights.

Two configuration requirements are mandatory and non-negotiable:

- **Scheduler**: `EulerDiscreteScheduler` with `timestep_spacing="trailing"`. The distillation training used trailing timestep spacing; a mismatch during inference leads to noticeably degraded output.
- **Guidance scale**: The distilled model has internalized the guidance effect into its weights, requiring no external guidance. This project sets `guidance_scale = 1.0`, i.e., **classifier-free guidance is completely disabled**.

> **Benefit of disabling CFG:** In Diffusers, when `guidance_scale > 1`, classifier-free guidance is enabled, and both the UNet and ControlNet must each run one forward pass with batch=2 (positive branch + negative branch). Since Lightning has already internalized the guidance effect through distillation, this extra computation is pure waste. Setting this value to `1.0` halves the batch size, directly reducing the denoising computation by approximately half, the single largest optimization in this project. The trade-off is that negative prompts become ineffective, a feature of limited value for a distilled model that has already internalized guidance.

4.4 Multi-LoRA Dynamic Blending

Leveraging PEFT's multi-adapter mechanism, the system registers the acceleration LoRA and all 8 style LoRAs into the pipeline at startup. At runtime, `set_adapters([accel_LoRA, style_LoRA], adapter_weights=[1.0, scale])` dynamically specifies the active combination and weights; the weight matrices are linearly combined and superimposed onto the base weights during the forward pass, requiring no disk I/O whatsoever.

#	Display Name	Style	LoRA Repository
1	Anime	Anime	`ntc-ai/SDXL-LoRA-slider.anime`
2	Pixel Art	Pixel Art	`ntc-ai/SDXL-LoRA-slider.pixel-art`
3	Studio Ghibli	Studio Ghibli	`ntc-ai/SDXL-LoRA-slider.Studio-Ghibli-style`
4	Cartoon	Cartoon	`ntc-ai/SDXL-LoRA-slider.cartoon`
5	Oil Painting	Oil Painting	`ntc-ai/SDXL-LoRA-slider.oil-painting`
6	Pixar	Pixar	`ntc-ai/SDXL-LoRA-slider.pixar-style`
7	Ultra-realistic Illustration	Ultra-realistic Illustration	`ntc-ai/SDXL-LoRA-slider.ultra-realistic-illustration`
8	2000s Indie Comic	2000s Indie Comic	`ntc-ai/SDXL-LoRA-slider.2000s-indie-comic-art-style`

The style collection is filtered by Hugging Face download count (descending), excluding non-art-style entries (quality enhancement, human pose, makeup, etc. sliders). Each style comes with a set of recommended prompts, auto-filled when the style is selected in the UI.

4.5 Key Parameters

Parameter	Default	Purpose
`strength`	0.99	Redraw intensity; also determines actual denoising steps `int(steps × strength)`
`controlnet_conditioning_scale`	0.4	Edge constraint strength; higher values adhere more closely to the original structure
`num_inference_steps`	8	Nominal sampling steps, matched to the Lightning 8-step LoRA
`guidance_scale`	1.0	CFG guidance coefficient; 1.0 disables CFG; the distilled model requires no external guidance
`seed`	42	Random seed; fixing it enables rigorous controlled style comparison experiments

5. AMD Radeon GPU / ROCm Adaptation

5.1 Runtime Environment

Accelerator

Item	Configuration
GPU	AMD Radeon Graphics × 1 (`MARKET_NAME` value reported by `amd-smi static`)
GFX Architecture	`gfx1100` (RDNA 3 / Navi 31, LLVM target `amdgcn-amd-amdhsa--gfx1100`)
VRAM Capacity	49,136 MB (approx. **48 GB**)
PCIe Location	`0000:03:00.0` (HIP Device 0)
Power Cap	241 W
VBIOS	`00162356`

Software Stack

Item	Configuration
ROCm Version	**7.2.1**
Kernel Driver	`amdgpu` 6.16.13
Management Tool	`amd-smi` 26.2.2 (+e1a6bc5663)
Deployment	Linux Baremetal (non-virtualized, GPU passed through to container)
PyTorch Version	**2.9.1** (built from source, commit `ff65f5b`)
Operating System	Ubuntu 24.04.4 LTS
Python	`/opt/venv` virtual environment inside container

Host Platform

Item	Configuration
CPU	AMD EPYC 9334 32-Core Processor × 2 sockets (Zen 4, Family 25 Model 17)
Cores / Threads	64 physical cores / 128 logical threads
NUMA Topology	2 nodes (node0: CPU 0—31, 64—95; node1: CPU 32—63, 96—127)
Cache	L2 64 MiB / L3 256 MiB (8 groups)
System Memory	503 GiB

> This project runs on a **full AMD platform**, with an AMD EPYC 9334 as the host CPU, an AMD Radeon GPU as the inference accelerator, and the ROCm 7.2.1 software stack. The CPU side's 128 threads and 503 GiB of memory provide ample resources for host-side tasks such as model weight loading and image encoding/decoding, and do not constitute a bottleneck.

5.2 Adaptation Strategy: Zero-code-change Migration

The ROCm build of PyTorch provides semantic-level mapping of the CUDA programming model through the HIP runtime; the `torch.cuda.*` API family directly targets the AMD GPU under a ROCm build (HIP Device 0 in this environment). Therefore, **all device-related code in this project** (including `.to("cuda")`, `torch.cuda.synchronize()`, `torch.Generator(device="cuda")`) **runs on the ROCm 7.2.1 platform without any modification**. The source code remains fully identical to the NVIDIA platform, with no platform-specific code branches or conditional compilation.

The value of this strategy is that the project can support both hardware platforms from a single codebase, reducing maintenance costs and facilitating cross-platform performance comparisons.

5.3 Precision and Numerical Stability Adaptation

This project uses fp16 end-to-end, a choice that directly aligns with the architectural characteristics of the target hardware: **gfx1100 (RDNA 3) introduced WMMA (Wave Matrix Multiply-Accumulate) matrix instructions, natively supporting fp16 matrix multiply-accumulate operations**. The vast majority of computation in diffusion models is concentrated in the convolutions and attention matrix multiplications of the UNet and ControlNet; running in fp16 directly hits this hardware acceleration path.

- **End-to-end fp16.** All model components (UNet, ControlNet, dual text encoders, VAE) are uniformly loaded with `torch_dtype=torch.float16`, fully leveraging RDNA 3's half-precision matrix compute and halving VRAM usage.
- **VAE overflow mitigation.** The official SDXL VAE has a known numerical overflow issue under fp16, producing black images or color blocks. This project uses `madebyollin/sdxl-vae-fp16-`

`fix`, whose weights have been retrained to accommodate the fp16 dynamic range, a necessary condition for viable end-to-end fp16 operation.

- **Base model fp16 variant.** Loading with `variant="fp16"` downloads half-precision weights directly, avoiding the extra overhead and VRAM spikes of runtime conversion.

5.4 VRAM Management

The system adopts a "fully resident, no offload" strategy:

- The SDXL backbone, distilled ControlNet, VAE, Lightning LoRA, and all 8 style LoRAs are fully resident in VRAM (Canny detection is a pure CPU operator and uses no VRAM);
- Each style LoRA is only 9 MB; 8 total approximately 72 MB, negligible relative to the base model, making the "full preload" design entirely viable in VRAM terms;
- `enable_model_cpu_offload()` is not used, avoiding per-step Host↔Device weight transfers; in PCIe bandwidth-constrained scenarios, the latency from offloading often exceeds the computation itself.

The 48 GB of VRAM on this platform provides ample headroom for this strategy: even with fp16 full precision, ControlNet, and all LoRAs resident, there remains substantial VRAM margin, eliminating any need to compromise for capacity (such as quantization, tiled decoding, or hierarchical offloading). This is also the direct premise for quantization being deemed "unnecessary" in Section 5.5.

Measured VRAM usage: **PyTorch tensor peak 9.91 GB, allocator reserved 11.21 GB** (see Section 5.6 for details). Relative to 48 GB of available capacity, the utilization rate is approximately 23%, with ample remaining headroom.

5.5 Quantization Trade-offs

The project implementation retains a 4-bit quantization path based on bitsandbytes NF4 (the `USE_NF4` switch in `gen_image.py`), which can compress UNet weights to 4-bit to reduce VRAM usage. However, after empirical evaluation, it is **disabled by default** for two reasons:

- **Mismatched benefit direction.** The core value of quantization is reducing VRAM usage, but this platform's 48 GB far exceeds the project's requirements (measured usage is only ~11 GB); VRAM has never been a constraint.
- **Negative net benefit.** 4-bit weights must be dequantized back to compute precision on every forward pass; under this project's workload characteristics, this overhead cannot be compensated by VRAM savings, and end-to-end latency actually increases.

This quantization path is retained in the code as a fallback for low-VRAM devices (e.g., 8—12 GB consumer GPUs), activatable via a single boolean switch without affecting the main path.

5.6 Performance

Test conditions: input image 1024×1024, `steps=8`, `strength=0.7` (5 actual denoising steps), `guidance_scale=1.0`, `controlnet_conditioning_scale=0.5`, `seed=42`. Data collected from steady-state operation after warmup, directly output by the system's built-in per-stage timing and VRAM statistics. (The UI defaults are `strength=0.99` / `ControlNet Scale=0.4`, per §4.5, which would run 7 actual steps; this measurement uses a 5-step baseline for comparability.)

Metric	Measured Value	Share	Notes
LoRA switch (`set_adapters`)	48.1 ms	2.3%	Pure in-VRAM weight coefficient adjustment, no disk I/O
Canny edge extraction	0.01 s	0.5%	OpenCV operator, CPU-side computation, no model weights
Setup (text encoding + 1st step)	0.60 s	29.3%	Dual CLIP encoding + VAE Encode + 1st denoising step
Per-step denoising time	**0.246 s**/step	48.0%	UNet + ControlNet single-step forward, CFG disabled (batch=1), steps 2—5 (4 steps)
VAE decoding	0.40 s	19.5%	fp16-fix VAE, latent → 1024×1024 image
Pipeline total time	1.98 s	96.6%	Encoding + 5 denoising steps + decoding
End-to-end per-image time	**2.05 s**	100%	Complete chain from request to image output
VRAM peak (PyTorch allocated)	9.91 GB	N/A	Actual tensor memory for model weights + intermediate activations
VRAM peak (PyTorch reserved)	11.21 GB	N/A	Allocator cache pool; `amd-smi` process-level readings add HIP runtime context overhead on top

> **Conclusion: 1024×1024 image-to-image end-to-end in 2.05 seconds**, achieving interactive-level responsiveness. The cost breakdown is: **denoising 77%** (Setup includes the first step), **VAE decoding 20%**, and LoRA switching + edge extraction combined under 3%.
>
> Two points are worth noting: First, **LoRA switching is only 48 milliseconds**, validating the effectiveness of the "full preload + weight blending" design: a traditional disk-reload approach would incur seconds of overhead, making rapid comparison across 8 styles infeasible. Second, **after compressing steps to just 5, the fixed overhead (encoding in Setup + VAE decoding, totaling ~0.75 s) is already approaching the magnitude of denoising itself**, meaning the marginal benefit of further reducing sampling steps is rapidly diminishing; future optimization efforts should shift toward the encoding/decoding stages (see Section 5.8).

5.7 Completed Platform-Side Optimizations

#	Optimization	Description
1	**8-step distilled sampling**	Lightning LoRA + trailing scheduler, sampling steps reduced from 50 to 8 (5 actual steps in image-to-image mode)
2	**LoRA VRAM-resident**	Zero disk I/O on style switch; only adapter weights adjusted
3	**End-to-end fp16**	Paired with fp16-fix VAE for numerical stability
4	**Startup warmup**	GPU kernel first-compilation overhead shifted to service startup; users hit steady-state performance on first request
5	**No offload, fully resident**	Eliminates per-step Host↔Device weight transfers
6	**CFG disabled**	`guidance_scale = 1.0`, UNet and ControlNet batch reduced from 2 to 1, denoising computation halved (see Section 4.3)
7	**Edge map caching**	Caches the most recent Canny edge map; skips edge detection when tuning parameters or switching styles on the same image. This mechanism was highly beneficial with the earlier HED neural-network detector; after switching to OpenCV Canny, per-extraction time dropped to 0.01 s, making this a marginal optimization
8	**MIOpen convolution algorithm auto-tuning**	Enables `torch.backends.cudnn.benchmark`, which maps to MIOpen's exhaustive find mode on ROCm, empirically selecting the optimal convolution algorithm for fixed input shapes. With resolution fixed at 1024×1024 and batch fixed at 1 after disabling CFG, shapes are fully stable, satisfying the precondition for this mode. The one-time tuning overhead is absorbed by startup warmup, and MIOpen persists results to a local database, eliminating re-tuning after service restarts

5.8 Future Optimization Directions

1. **Lightweight VAE decoding**: measurements show VAE decoding accounts for ~20% of end-to-end time; with total steps compressed to 5, it is now the second-largest cost after denoising. TAESD could reduce this stage from hundreds of milliseconds to tens of milliseconds, or a two-stage strategy could be used: TAESD for fast preview, full VAE for final output upon user confirmation.
2. **Text embedding caching**: skip dual CLIP encoding when the same prompt is reused. This overhead is included in Setup and accounts for ~17% of end-to-end time.
3. **Disable timing synchronization**: remove per-step `torch.cuda.synchronize()` in production mode. This synchronization is used for precise per-step timing collection but interrupts the CPU/GPU pipeline; it can be disabled once performance data collection is complete.
4. **More aggressive distillation tier**: switch to the 4-step Lightning LoRA, further reducing actual denoising steps from 5 to 2—3; the acceptable quality loss must be evaluated.
5. **torch.compile**: apply graph compilation to the UNet and ControlNet, typically yielding 20—30% gains. Two points must be verified first: whether dynamic `set_adapters` switching triggers recompilation, and the maturity of the Triton backend on gfx1100 (RDNA 3), where optimization on CDNA architectures is more mature.

> It should be noted that **this project does not consider any optimization aimed at reducing VRAM usage** (quantization, CPU offload, tiled VAE decoding, etc.). With 48 GB of VRAM and measured usage of ~11 GB, such approaches offer no benefit space and generally come at the cost of speed. Optimization efforts are focused on reducing computation and improving execution efficiency.